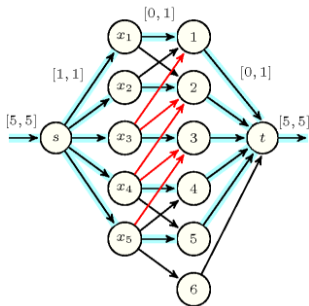


L03: Global Constraints



Prof. Tias Guns and Dr. Hendrik 'Henk' Bierlee

KU LEUVEN

Partly based on slides from Pierre Flener and Dimos Tsouros.

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and NVALUEEQ()

COUNTSEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

First: primitive constraints and propagation

From Lecture 1:

Solving combinatorial problems **intelligently**:

- ▶ **Search**: Explore the space of possible assignments.
- ▶ **Inference**: Reduce the space to feasible (partial) assignments.

In this lecture we refine *how* solvers do inference:

First: primitive constraints and propagation

From Lecture 1:

Solving combinatorial problems **intelligently**:

- ▶ **Search**: Explore the space of possible assignments.
- ▶ **Inference**: Reduce the space to feasible (partial) assignments.

In this lecture we refine *how* solvers do inference:

Definition

Primitive constraint: a constraint that the solver can do **inference** with.

For CP, PB and SAT inference=**propagation**.

Definition

Propagation: using the properties of a primitive constraint to **reduce the domains** of participating decision variables by removing unsupported values.

Example: $x + 2 \cdot y \geq 3$, $D(x) = \{0, 1, 2\}$, $D(y) = \{0, 1, 2\}$
 $\implies D(y) = \{\cancel{0}, 1, 2\}$

Solvers and primitive constraints

Different solver paradigms have different **primitive** constraints:

- ▶ **Boolean SATisfiability solvers:** Clause: $b_1 \vee b_2 \vee \dots \vee b_n$
- ▶ **Pseudo-Boolean solvers:** $\sum w \cdot b \leq c$
(with w, c constant, $\leq \in \{=, <, \leq, >, \geq\}$)
- ▶ **Integer Linear solvers:** $\sum w \cdot v \leq c$ and $v \leq c$
(with w, c constant, $\leq \in \{=, \leq, \geq\}$)

CP solvers have all-of-the-above + $\neq$ + *global constraints*

Definition

Global Constraint: a named constraint that represents a non-trivial relationship between decision variables.

They are typically used to represent a specific combinatorial substructure commonly found in constraint satisfaction problems.

Example

Well-known global constraints

- ▶ ALLDIFFERENT()
- ▶ CIRCUIT()
- ▶ NOOVERLAP()
- ▶ ...

Modeling with global constraints

When you write a model with global constraints:

1. If the solver has this global constraint as a **primitive constraint**: it will use its propagator to eliminate values during search
2. If the solver does not have it as primitive constraint: the constraint needs to be **decomposed** and rewritten into primitive constraints. This often involves additional 'auxiliary' decision variables.

Definition

Decomposition: rewriting a constraint into an equivalent set of simpler constraints

A constraint modeling system will, for a chosen target solver, decompose and rewrite constraints until only primitive constraints remain.

Why use Global Constraints?

+ **Expressiveness!**

More compact and intuitive models, closer to problem definition.

Many expressive global constraints available

- ▶ **Simplified modeling:** Global constraints enable the modeler to express complex conditions simpler.
Simpler modeling reduces the chance of modeling errors.
- ▶ **Compactness:** Instead of writing multiple smaller constraints, a single global constraint can capture the entire logic.
Makes the model more intuitive and readable

Example

Task allocation: I want all tasks to be allocated to a different team.

▶ *Without global:*

$Task_0 \neq Task_1, Task_0 \neq Task_2, Task_1 \neq Task_2, \dots$

▶ *With global:* ALLDIFFERENT(Task)

Why use Global Constraints? – CPMpy

+ **Expressiveness!**

More compact and intuitive models, closer to problem definition.

Many expressive global constraints available

- ▶ **Simplified modeling:** Global constraints enable the modeler to express complex conditions simpler.
Simpler modeling reduces the chance of modeling errors.
- ▶ **Compactness:** Instead of writing multiple smaller constraints, a single global constraint can capture the entire logic.
Makes the model more intuitive and readable

Example

Task allocation: I want all tasks to be allocated to a different team

▶ *Without global:*

`Task[0] != Task[1], Task[0] != Task[2], Task[1] != Task[2], ...`

▶ *With global:* `cp.AllDifferent(Task)`

Why use Global Constraints?

- + **Efficiency!** (If available as primitive constraint)
Faster solving, due to better **inference** (stronger **propagation**), enabled by more global information in the model
 - ▶ More global information in one constraint can result to advanced filtering of the **search** space
 - ▶ Specialized algorithms to detect **conflicts** faster during search.

Example

Task allocation: I want to allocate n tasks to m teams, s.t. each task is assigned to a different team. Assume we have $m < n$.

- ▶ *Without global:* Each $\neq$ constraint needs 2 values (teams) available for its tasks. Will realize that there are not enough values only after extensive search of possible assignments
- ▶ *With global:* ALLDIFFERENT(Task) will directly recognise that we cannot put m different values (teams) in n variables (tasks) if we have $m < n$ during search.
← *Pigeonhole principle*

Modelling with Global Constraints:

Many global constraints exist, capturing different combinatorial properties:

Global-Constraint Catalogue <https://sofdem.github.io/gccat>

Functional Global Constraints: Global constraints that have a functional component, such as `MINIMUMEQ()`, `MAXIMUMEQ()`, `COUNTSEQ()`, `NVALUEEQ()`, etc.

Definition

A global constraint $G(V)$ is *functional* if and only if there exists a partitioning of the arguments V into two non-empty and non-overlapping subsets $V1$, $V2$, such that *the assignment of decision variables in subset $V2$ is defined using a function on the subset $V1$.*

Modelling with Global Constraints:

Definition

A global constraint $G(V)$ is *functional* if and only if there exists a partitioning of the arguments V into two non-empty and non-overlapping subsets $V1$, $V2$, such that *the assignment of variables in subset $V2$ is defined using a function on the subset $V1$.*

In many cases, this involves associating the value of the functional component with a variable:

Examples

- ▶ $\text{MAXIMUMEQ}(X, v)$ expresses that $v = \text{MAXIMUM}(X)$. Now, v can be used in other expressions.
- ▶ $\text{MINIMUMEQ}(X, v)$ expresses that $v = \text{MINIMUM}(X)$.
- ▶ ...

Modelling with Global Constraints – CPMpy

- ▶ A number of commonly-used **global constraints** are available in CPMpy: `AllDifferent`, `AllEqual`, `Cumulative`, `Table`, ... API documentation: <http://cpmpy.readthedocs.io/en/latest/api/expressions/globalconstraints.html>
- ▶ All global constraints can be reified - be nested in other expressions: e.g. `cp.sum(cp.AllDifferent(x), cp.AllDifferent(y), cp.AllDifferent(z)) > 2`
- ▶ A number of commonly-used **global functions** are available too, representing only the *functional* component: e.g. `cp.Maximum(X)`
 - ▶ Can be used nested in any expression: e.g. `cp.Maximum(X) < 10`
 - ▶ Several Global functions available: `Maximum`, `Minimum`, `Count`, `NValue`, ... API documentation: <http://cpmpy.readthedocs.io/en/latest/api/expressions/globalfunctions.html>

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and NVALUEEQ()

COUNTSEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and **NVALUEEQ()**

COUNTEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

ALLDIFFERENT()

Definition (Laurière, 1978)

The ALLDIFFERENT(X) constraint holds if and only if all the elements of array X take distinct values.

Its **decomposition** is a conjunction of $\frac{n \cdot (n-1)}{2}$ disequality constraints when X has n elements:

$$X_i \neq X_j \quad \forall i, j \in \{1, \dots, n\}, i < j$$

Examples

- Sudoku, Room assignment, Task allocation, ...

ALLDIFFERENT()

Definition (Laurière, 1978)

The ALLDIFFERENT(X) constraint holds if and only if all the elements of array X take distinct values.

Its **decomposition** is a conjunction of $\frac{n \cdot (n-1)}{2}$ disequality constraints when X has n elements:

$$X_i \neq X_j \quad \forall i, j \in \{1, \dots, n\}, i < j$$

Examples

- Sudoku, Room assignment, Task allocation, ...

Variant: The ALLDIFFERENTEXCEPTN(X, N) constraint allows multiple occurrences of the excepted value N only (e.g. except 0).

ALLDIFFERENT() – CPMpy

Definition (Laurière, 1978)

The ALLDIFFERENT(X) constraint holds if and only if all the elements of array X take distinct values.

Its decomposition is a conjunction of $\frac{n \cdot (n-1)}{2}$ disequality constraints when X has n elements:

Math: $X_i \neq X_j \quad \forall i, j \in \{1, \dots, n\}, i < j$
CPMpy: `model.add( var1 != var2 for var1, var2 in all_pairs(X) )`

Variant: The `cp.AllDifferentExceptN(X, N)` constraint allows multiple occurrences of the excepted value N only (e.g. except 0).

Example

Sudoku: we want distinct values in rows, columns and blocks.

Using the ALLDIFFERENT(X) global constraint:

$\text{AllDifferent}(\{G_{ij} \mid j \in \{1, \dots, 9\}\}) \quad \forall i \in \{1, \dots, 9\} \quad (\text{rows})$

$\text{AllDifferent}(\{G_{ij} \mid i \in \{1, \dots, 9\}\}) \quad \forall j \in \{1, \dots, 9\} \quad (\text{columns})$

$\text{AllDifferent}(\{G_{kl} \mid k \in \{i, \dots, i+2\}, l \in \{j, \dots, j+2\}\}) \quad \forall i, j \in \{1, 4, 7\} \quad (\text{blocks})$

The above is more expressive (and efficient) than using binary not equal constraints:

$G_{ij} \neq G_{ik} \quad \forall i \in \{1, \dots, 9\}, \forall j, k \in \{1, \dots, 9\}, j < k \quad (\text{rows})$

$G_{ij} \neq G_{kj} \quad \forall j \in \{1, \dots, 9\}, \forall i, k \in \{1, \dots, 9\}, i < k \quad (\text{columns})$

$G_{kl} \neq G_{mn} \quad \forall i, j \in \{1, 4, 7\}, \forall k, m \in \{i, \dots, i+2\},$
 $\forall l, n \in \{j, \dots, j+2\}, (k, l) < (m, n) \quad (\text{blocks})$

Example (CPMpy (indexing offset 0))

Sudoku: we want distinct values in rows, columns and blocks.

```
1 [cp.AllDifferent(row) for row in grid],
2 [cp.AllDifferent(col) for col in grid.T], # numpy's Transpose
3 [cp.AllDifferent(grid[i:i+3, j:j+3]) \
4   for i in range(0, 9, 3) for j in range(0, 9, 3)],
```

The above is more expressive (and efficient) than using binary not equal constraints:

```
1 [[cell1 != cell2 for cell1, cell2 in all_pairs(row)]
2   for row in grid],
3 [[cell1 != cell2 for cell1, cell2 in all_pairs(col)]
4   for col in grid.T],
5 [[cell1 != cell2 for cell1, cell2
6   in all_pairs(grid[i:i+3, j:j+3].flat)]
7   for i in [0, 3, 6] for j in [0, 3, 6]]
```

ALLDIFFERENT(), combinatorial substructure

What combinatorial substructure does the ALLDIFFERENT(X) global constraint represent?

A matching problem!

Let $|X| = 5$ and let

$$D(X_1) = \{1, 2\},$$

$$D(X_2) = \{1, 2\},$$

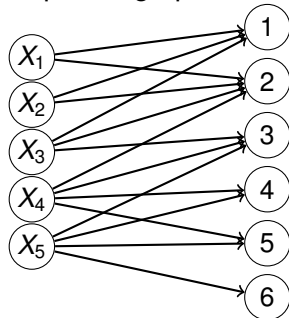
$$D(X_3) = \{1, 2, 3\},$$

$$D(X_4) = \{2, 3, 4, 5\},$$

$$D(X_5) = \{3, 4, 5, 6\}.$$

So ALLDIFFERENT(X) states that we have to match every X_i with a distinct value.

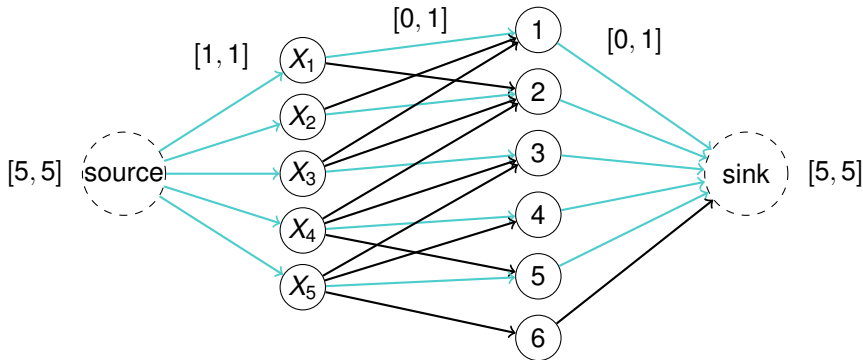
Bipartite graph view:



ALLDIFFERENT(), efficient checking

How to efficiently compute if an assignment is still possible? If there exists a matching?

Can use highly efficient algorithms for matching, e.g. have the **propagator** solve a *network flow* problem over the bipartite graph; with $[\min, \max]$ allowed flow on every edge: **solution** = flow of 1 for **cyan edges**)

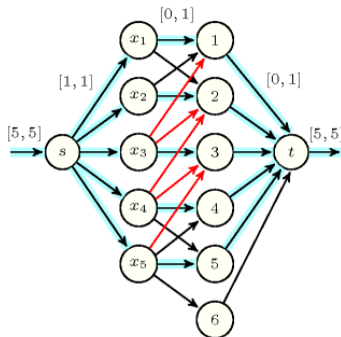


ALLDIFFERENT(), better propagator

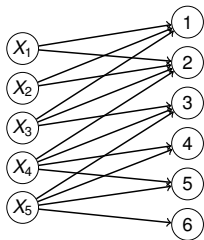
- + **Efficiency!** Better propagation in the primitive constraint due to capturing global properties

Using a flow model for ALLDIFFERENT() propagation

- ▶ feasible flows in the flow model = solutions to the constraint
- ▶ detect arcs that cannot carry flow in any feasible solutions $\rightarrow$ remove values from the domains of variables
- ▶ **Blue** arcs represent a maximal matching, **red** arcs represent infeasible arcs
- ▶ **Detect inconsistency early:** flow from (some) variables **cannot** be directed through the available values
 - ▶ Not detected early through binary constraints



Linear decomposition of AllDifferent



There is a natural *linear* decomposition that corresponds to the bipartite matching formulation (and the flow graph):

Each arc (X_i, v) corresponds to the expression $(X_i = v)$: true when decision variable X_i takes value v , false otherwise. Let $V = \bigcup_{X_i \in X} D(X_i)$.

$$\sum_{X_i \in X} \llbracket X_i = v \rrbracket \leq 1 \quad \forall v \in V \quad (\text{each value used at most once})^1$$

While this is an efficient, more linear-friendly decomposition than the integer $\neq$ decomposition, it leads to weaker propagation than the global propagator that actively searches for unreachable arcs.

An integer variable already takes exactly one domain value, so $\sum_{v \in D(X_i)} \llbracket X_i = v \rrbracket = 1$ needs no constraint.

Linear decomposition of AllDifferent – CPMpy

Let $V = \bigcup_{X_i \in X} D(X_i)$.

Math:
$$\sum_{X_i \in X} \llbracket X_i = v \rrbracket \leq 1 \quad \forall v \in V$$

Iverson brackets $\llbracket \cdot \rrbracket$ turn a true/false condition into a number: 1 if it holds, 0 if not.

CPMpy:

```
1 for v in V:
2     m.add( cp.sum(xi == v for xi in X) <= 1 )
```

Note how in math we write $X_i = v$ (single equality sign) and need Iverson brackets when writing Boolean expressions in mathematical operators;

In CPMpy we write `xi == v` (double equality sign) and `cp.sum(...)` is also defined on Boolean expressions.

NVALUEEQ()

Definition (Pachet and Roy, 1999)

The NVALUEEQ(X , res) functional global constraint holds if and only if the number of *unique* values taken inside array X is equal to res .

If the argument X is a 1-dimensional array of length n , then this means:

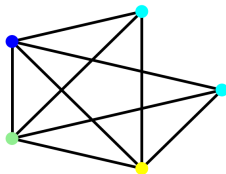
$$|\{X_1, \dots, X_n\}| = res$$

Let V be the set of unique values in the domains of X .

The **decomposition** creates an auxiliary Boolean variable B_v , indicating if that value is used, and constrains their count:

$$B_v = \bigvee_i (X_i = v) \qquad \forall v \in V$$
$$res = \sum_v B_v$$

NVALUEEQ() example



Example

Graph colouring: Neighbouring nodes must be given distinct colours + minimize the number of colours, i.e., minimize the number of *unique* values used:

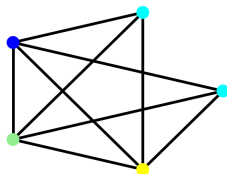
$\text{node}_1 \neq \text{node}_2,$

$\forall (\text{node}_1, \text{node}_2) \in \text{Edges}$

$\text{NVALUEEQ}(\text{nodes}, \text{res})$

minimize *res*

NVALUEEQ() example – CPMpy



Example

Graph colouring: Neighbouring nodes must be given distinct colours + minimize the number of colours, i.e., minimize the number of *unique* values used:

```
1 # Adjacent vertices must have different colours
2 model.add([nodes[i] != nodes[j] for (i, j) in Edges])
3 # Minimize nr of unique colours used
4 model.minimize(cp.NValue(nodes))
```

NVALUEEQ() vs ALLDIFFERENT()

Recap NVALUEEQ(): if the argument X is a 1-dimensional array of length n , then NVALUEEQ(X , res) expresses:

$$|\{X_1, \dots, X_n\}| = res$$

If $res = n$ (the *length* of array X), then NVALUEEQ(X , res) is equivalent to ALLDIFFERENT(X).

Note, the decomposition would needlessly create the auxiliary B_v variables (while we don't need $\bigvee_i (X_i = v)$ as we know a value will either not be used, or be used exactly once).

In general, always use the most specific available constraint predicate!

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and NVALUEEQ()

COUNTSEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

COUNT EQ()

Definition

The COUNT EQ(X , val , res) functional global constraint holds if and only if the number of *occurrences* of value val in array X is equal to res .

Note: val can also be a decision variable (the constraint will hold for its value).
The **decomposition** is the following:

$$res = \sum_i \llbracket X_i = val \rrbracket$$

Examples

COUNT([1, 1, 2, 3, 2], 1) = 2, COUNT([1, 1, 2, 3, 2], 3) = 1

► Doctor rostering, Job allocation, Timetabling, ...

COUNTEQ() – CPMpy

Definition

The COUNTEQ(X , val , res) functional global constraint holds if and only if the number of *occurrences* of value val in array X is equal to res .

Note: val can also be a decision variable (the constraint will hold for its value).

The **decomposition** is the following:

$$res == \text{cp.sum}(X == val)$$

Examples

- ▶ Doctor rostering, Job allocation, Timetabling, ...

COUNT_EQ() example – CPMpy

Example (CPMpy model for our doctor rostering (indexing offset 0))

```
1 roster = cp.intvar(0,n_shifts-1, shape=(n_doctors,n_days))
2 model = cp.Model(
3     # on-call/day = 1
4     [cp.Count(roster[:,d], Call) == 1 for d in range(n_days)],
5     # oper/weekday <= 2; assume d mod 7 == 0 for Monday, etc
6     [cp.Count(roster[:,d], Oper) <= 2 for d in range(n_days) if d % 7 <= 4],
7     # oper/week >= 7
8     [cp.Count(roster[:,s:s+7], Oper) >= 7 for s in range(0, n_days, 7)],
9     # appt/week >= 4
10    [cp.Count(roster[:,s:s+7], Appt) >= 4 for s in range(0, n_days, 7)],
11    # free after oper
12    [(roster[p,d] == Oper).implies(roster[p,d+1] == Free) for p in range(n_doctors
13    ) for d in range(n_days-1)],
14    # maximize nr of free shifts in weekend
15    model.maximize(cp.sum([cp.Count(roster[:,s+5:s+7], Free) for s in range(0, n_days,
16    7)]))
17 model.solve()
```

GLOBALCARDINALITYCOUNT()

What if you want to count multiple values at once?

Definition (Régim, 1996)

The GLOBALCARDINALITYCOUNT(X , $Vals$, Res) global constraint holds if and only if for each value $Vals_i$, the number of occurrences in array X is equal to Res_i .

Note: $Vals$ can also be an array of decision variables.

The **decomposition** is the following:

$$\text{COUNT}EQ(X, Vals_i, Res_i) \quad \forall i \in \{1, \dots, |Vals|\}$$

Q: Why/when is it better to use GCC over Count?

GLOBALCARDINALITYCOUNT()

What if you want to count multiple values at once?

Definition (Régim, 1996)

The GLOBALCARDINALITYCOUNT(X , $Vals$, Res) global constraint holds if and only if for each value $Vals_i$, the number of occurrences in array X is equal to Res_i .

Note: $Vals$ can also be an array of decision variables.

The **decomposition** is the following:

$$\text{COUNT}EQ(X, Vals_i, Res_i) \quad \forall i \in \{1, \dots, |Vals|\}$$

Q: Why/when is it better to use GCC over Count?

Examples

- Nurse rostering, facility location, car sequencing, ...

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and NVALUEEQ()

COUNTSEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

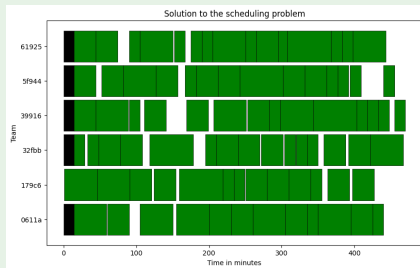
5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

Example (Scheduling aircraft assembly transports (Airbus))



Constraints to be satisfied:

1. Each task has a pre-determined duration and allowed time-window
2. Each task can only be handled by a subset of worker teams
3. Some tasks have *precedence* relations (e.g. same aircraft)
4. ...

Objective function to be minimised:

Minimize number of teams needed, and the workload balance between teams.

Scheduling tasks

Scheduling involves *allocating tasks in time*.

In time = tasks should have a start-time and an end-time (and hence a duration).

Definition (Task)

A task T is defined as a tuple $T = \langle s, d \rangle$ where:

- ▶ s is the starting time of task T
- ▶ d is the duration of task T

One can compute the time when it has just ended as $e = s + d$.



Some CP solvers have a special task decision variable, or even *optional* task decision variables (e.g. if it can be executed by multiple teams/machines).

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and NVALUEEQ()

COUNTSEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

Scheduling with NoOverlap

Basic setting: schedule tasks for a *single* team, where tasks can not overlap.

Definition (Carlier, 1982)

The NOOVERLAP(S, D) global constraint, where each task T_i has start-time S_i and duration D_i , holds if and only if no two tasks T_i and T_j overlap in time.

Hence for every pair, either T_i ends before T_j starts, or T_i starts after T_j ends.

The **decomposition** is the following:

$$(S_i + D_i \leq S_j) \vee (S_j + D_j \leq S_i) \qquad \forall i, j \text{ with } i < j$$

This constraint is also called the `Disjunctive` global constraint in the literature.

Scheduling with NoOverlap – CPMpy

Basic setting: schedule tasks for a *single* team, where tasks can not overlap.

Definition (Carlier, 1982)

The NOOVERLAP(S, D) global constraint, where each task T_i has start-time S_i and duration D_i , holds if and only if no two tasks T_i and T_j overlap in time.

Hence for every pair, either T_i ends before T_j starts, or T_i starts after T_j ends.

The **decomposition** is pairwise non-overlap:

```
for i,j in all_pairs(range(n)):
    model += (S[i] + D[i] <= S[j]) | (S[j] + D[j] <= S[i])
```

This constraint is also called the `Disjunctive` global constraint in the literature.

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and NVALUEEQ()

COUNTSEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

Scheduling tasks

What if tasks *can* overlap (e.g. be executed in parallel), but only up to some *capacity limit* of the machine?

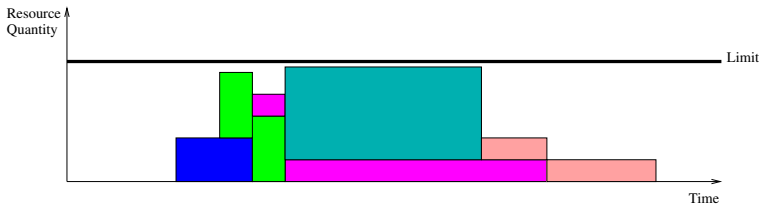


Fig: parallel tasks and a capacitated global reusable resource.

Requires the concept of *resource usage* R_i for each task T_i .

Examples

- Memory use in cloud scheduling, room size in exam scheduling, oven capacity in metal processing, reservoir size in water management, ...

CUMULATIVE() resource use

Definition (Aggoun and Beldiceanu, 1993)

The CUMULATIVE(S, D, R, c) global constraint, where each task T_i has start-time S_i , duration D_i and resource usage R_i , holds if and only if the total resource usage at any time does not exceed the capacity c .

The time-based **decomposition** is the following:

$$\sum_i R_i \cdot \mathbb{I}[(S_i \leq t) \wedge (t < S_i + D_i)] \leq c \quad \forall t \in \{\min(S_i)..\max(S_i + D_i)\}$$

The task-based **decomposition** is (only check each start time):

$$R_j + \sum_{i, i \neq j} R_i \cdot \mathbb{I}[(S_i \leq S_j) \wedge (S_j < S_i + D_i)] \leq c \quad \forall j$$

CUMULATIVE() resource use

Definition (Aggoun and Beldiceanu, 1993)

The CUMULATIVE(S, D, R, c) global constraint, where each task T_i has start-time S_i , duration D_i and resource usage R_i , holds if and only if the total resource usage at any time does not exceed the capacity c .

The time-based **decomposition** is the following:

$$\sum_i R_i \cdot \llbracket (S_i \leq t) \wedge (t < S_i + D_i) \rrbracket \leq c \quad \forall t \in \{min(S_i)..max(S_i + D_i)\}$$

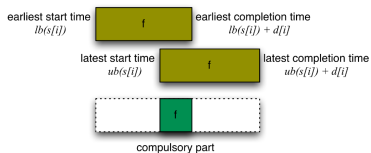
The task-based **decomposition** is (only check each start time):

$$R_j + \sum_{i, i \neq j} R_i \cdot \llbracket (S_i \leq S_j) \wedge (S_j < S_i + D_i) \rrbracket \leq c \quad \forall j$$

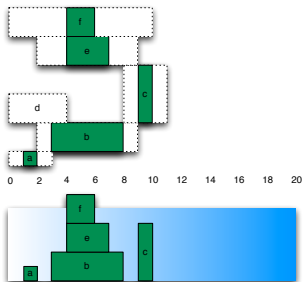
Q: If your solver does not have a CUMULATIVE(), when should you use which decomposition?

Propagating CUMULATIVE(), illustration²

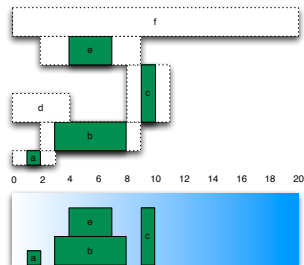
For each task, determine *compulsory time intervals*:



Fail if too much compulsory usage at some time interval:



Reduce start/end times based on compulsory intervals and capacity:



²Images courtesy of Peter Stuckey

Propagating CUMULATIVE()

The goal of the propagator is to ensure that

$$\sum_i R_i \cdot \mathbb{I}[S_i \leq t < S_i + D_i] \leq c$$

holds for all times t , *without explicitly enumerating each t .*

- ▶ **Core idea:** reason about *time intervals* and *task overlaps*
 - ▶ Compute **compulsory usage**: intervals of a task where it must occur in any feasible schedule
 - ▶ Use these to determine intervals where capacity is partially consumed
- ▶ **Propagation steps:**
 1. Detect overload: if the sum of compulsory usages in an interval exceeds c , fail.
 2. Tighten start and end times using reasoning on compulsory intervals, duration and remaining capacity.
 3. Optionally use **edge-finding** or **energetic reasoning** for stronger pruning.

Global constraint relations

$$\text{CUMULATIVE}(S, D, [1, 1, \dots, 1], 1) = ?$$

Global constraint relations

$$\text{CUMULATIVE}(S, D, [1, 1, \dots, 1], 1) = \text{NOOVERLAP}(S, D)$$

$$\text{CUMULATIVE}(S, [1, 1, \dots, 1], [1, 1, \dots, 1], 1) = ?$$

Global constraint relations

$$\text{CUMULATIVE}(S, D, [1, 1, \dots, 1], 1) = \text{NoOVERLAP}(S, D)$$

$$\begin{aligned}\text{CUMULATIVE}(S, [1, 1, \dots, 1], [1, 1, \dots, 1], 1) &= \text{NoOVERLAP}(S, [1, 1, \dots, 1]) \\ &= \text{ALLDIFFERENT}(S)\end{aligned}$$

In general, always use the most specific available constraint predicate!

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and NVALUEEQ()

COUNTSEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and NVALUEEQ()

COUNTSEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

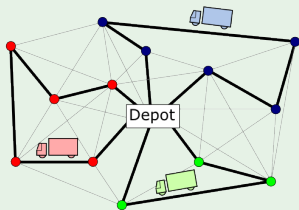
Enabling the representation of a circuit in a digraph

- ▶ Let decision variable S_v denote the successor of vertex v in the circuit.
- ▶ The domain of S_v is the set of vertices to which there is an arc from vertex v .

Definition (Laurière, 1978; Beldiceanu and Contejean, 1994)

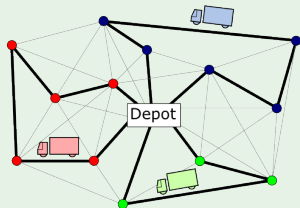
The CIRCUIT(S) global constraint holds if and only if $\forall v$ the arcs $v \rightarrow S_v$ form a Hamiltonian circuit: each vertex is visited exactly once.

Example (Vehicle Routing)



- ▶ Find optimal routes for multiple vehicles visiting a set of locations.
- ▶ 1 vehicle = Traveling Salesman Problem.

Example (Vehicle routing)



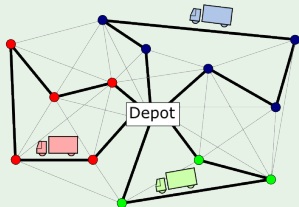
- ▶ Find optimal routes for multiple vehicles visiting a set of locations
- ▶ 1 vehicle = Traveling Salesman Problem

Travelling salesman problem (generalise this for vehicle routing problems with multiple vehicles or with side constraints):

Circuit(S)

$$\text{Minimize } \sum_{v=0}^{n-1} \text{distance}(v, S_v)$$

Example (Vehicle routing)



- Find optimal routes for multiple vehicles visiting a set of locations
- 1 vehicle = Traveling Salesman Problem

Travelling salesman problem (generalise this for vehicle routing problems with multiple vehicles or with side constraints):

Example (CPMpy Circuit TSP (indexing offset 0))

```
1 S = cp.intvar(0, cities - 1, shape=cities)
2 model = cp.Model()
3
4 model.add(cp.Circuit(S))
5 model.minimize(cp.sum(distance[city, S[city]] for city in range(
    cities)))
```

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and NVALUEEQ()

COUNTEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and NVALUEEQ()

COUNTSEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

ELEMENTEQ()

Modeling an **unknown element of an array**: $Arr[x]$ where x is a decision variable.

Example (Job allocation at minimal salary cost)

Given the salaries of work applicants *Salary* for different jobs,
find a work applicant for each job **such that** some constraints (on the qualifications of the work applicants for the jobs, on workload distribution, etc) are satisfied and the **total salary cost is minimal**:

$$workers_j \in \{0, \dots, Apps - 1\} \quad \forall j$$

$$total_cost = \sum_j Salary[workers_j]$$

minimize total_cost

We do not know at modelling time the worker allocated to each job!

ELEMENTEQ()

Modeling an **unknown element of an array**: $Arr[x]$ where x is a decision variable.

Example (Job allocation at minimal salary cost (indexing offset 0))

Given the salaries of work applicants *Salary* for different jobs,
find a work applicant for each job **such that** some constraints (on the qualifications of the work applicants for the jobs, on workload distribution, etc) are satisfied and the **total salary cost is minimal**:

```
# Objective: Minimize the total salary cost for assigned jobs
Salary = cp.cpm_array(Salary) # convert into a cpmPy-compatible array
total_cost = cp.sum([Salary[worker] for worker in workers])
model.minimize(total_cost)
```

We do not know at modelling time the worker allocated to each job!

We need to express the relation between variables/expressions and values of arrays based on unknown indices ...

The ELEMENTEQ() Global Constraint

We need to express the relation between variables/expressions and values of arrays based on unknown indices ...

Definition (Van Hentenryck and Carillon, 1988)

The ELEMENTEQ(*Arr*, *idx*, *res*) functional global constraint holds if and only if $Arr[idx] = res$.

For better model readability, the ELEMENTEQ() predicate is typically not directly used, when modeling languages allow direct indexing; e.g. `res == Arr[idx]` even if `idx` is an integer expression involving at least one decision variable.

Note that ELEMENT() can be multi-dimensional: `Arr[idx1, idx2]`.

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and NVALUEEQ()

COUNTSEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

TABLE

Definition

The $\text{TABLE}(X, T)$ constraint holds iff the values of array X of decision variables form a row in the n -by- $|X|$ matrix T of values.

Mathematically, $\exists \text{ row} \in T$ such that $\forall j, X_j = \text{row}_j$

Example

Assigning Workers W_1, W_2 to Shifts, but only the specific assignments are allowed:

$$T = \{(1, 2), (1, 3), (2, 3)\}$$

The $\text{TABLE}()$ constraint is applied as:

$$\text{TABLE}([W_1, W_2], T)$$

TABLE

Definition

The TABLE(X , T) constraint holds iff the values of array X of decision variables form a row in the n -by- $|X|$ matrix T of values.

Mathematically, $\exists \text{ row} \in T$ such that $\forall j, X_j = \text{row}_j$

One **decomposition** creates a Boolean variable per row, enforcing at least one true:

$$\begin{aligned} B_i &\rightarrow \bigwedge_j (X_j = T_{ij}) && \forall i \\ \bigvee_i B_i \end{aligned}$$

An alternative **decomposition** creates one integer decision variable $r \in [1..n]$ that represents the active row index, and posts an ELEMENT per X variable/column:

$$X_j = \text{ELEMENT}([T_{1j}, \dots, T_{nj}], r) \quad \forall j$$

Outline

1. Motivation and definition

2. Counting over integer values

ALLDIFFERENT() and NVALUEEQ()

COUNTEQ() and GLOBALCARDINALITYCOUNT()

3. Scheduling

NOOVERLAP()

CUMULATIVE()

4. Routing

CIRCUIT()

5. Ad-hoc constraints over integer values

ELEMENTEQ()

TABLE

6. Other global constraints

Other global constraints

- ▶ Non-linear arithmetic: `MINIMUMEQ()`, `MAXIMUMEQ()`, `ABSEQ()`
- ▶ Other Boolean operators: `XOR()`
- ▶ Other ad-hoc constraints: `SHORTTABLE()`, `NEGATIVETABLE()`
- ▶ Symmetry breaking (later lecture): `INCREASING()`, `INCREASINGSTRICT()`, `DECREASING()`, `DECREASINGSTRICT()`, `LEXLESS()`, `LEXLESSEQ()`, ...
- ▶ and more

CP modeling systems allow you to define your own global constraints and corresponding decomposition (= like a helper function)

CP solvers allow you to implement your own global constraints and corresponding propagator (= algorithms that speed up solving)

Summary

Global Constraints: captures combinatorial substructure in the problem

- ▶ Expressiveness: modeler can reuse existing building blocks
- ▶ Efficiency: solvers with matching primitive constraints could perform better, because:
 - ▶ it might infer *more* than the decomposition
 - ▶ it might avoid creating (many) auxiliary variables
 - ▶ it might use optimized data structures that make the check faster
- ▶ Compatibility: for other solvers, a known (ideally well-performing) default decomposition will be used. Sometimes even different decompositions depending on the target solver.
Could be faster than what you would write!